



Sprint 0 Product BackLog & UserHistory

Juan Camilo Cespedes Correal

Juan Camilo Zabala Ceron

David Felipe Moreno García

Brayhan Alexander Suárez Merchan

Universidad Manuela Beltrán

Carlos Eduardo Mujica Reyes

Arquitectura de Software 2026

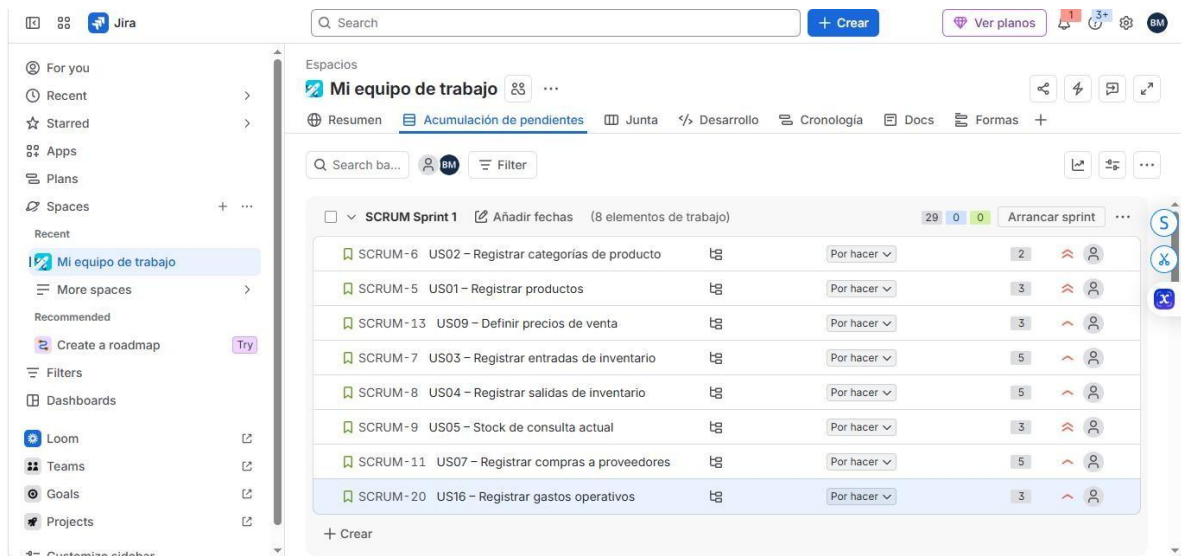
1. Product Backlog

Para la definición del Product Backlog del proyecto **ERP La 74**, el equipo elaboró un listado completo de historias de usuario a partir de los requerimientos funcionales (RF01RF14) definidos previamente, distribuidas en los cinco módulos del sistema: Inventario, Costos y Ventas, Empleados, Gastos y Reportes. En total se identificaron 23 historias de usuario.

El backlog fue priorizado utilizando dos técnicas complementarias:

- **MoSCoW** (Must, Should, Could, Won't), que permitió clasificar cada historia según su nivel de obligatoriedad para el funcionamiento mínimo del sistema.
- **Valor vs. Esfuerzo**, que permitió evaluar cada historia según el beneficio que aporta al negocio frente a la complejidad técnica de implementarla, identificando así los "Quick Wins" (alto valor, bajo esfuerzo) que debían priorizarse en los primeros sprints.

Con base en estas dos priorizaciones, el backlog fue ordenado colocando primero las historias clasificadas como Must con menor esfuerzo, seguidas de las Should y finalmente las Could, lo que garantiza que el equipo entregue primero el valor esencial del negocio. El listado completo, ordenado y priorizado, se encuentra documentado en Jira (sección Backlog) y se adjunta como evidencia en el Anexo



For you

Recent

Starred

Apps

Plans

Spaces

Recent

Mi equipo de trabajo

More spaces

Recommended

Create a roadmap

Filters

Dashboards

Loom

Teams

Goals

Projects

Customize sidebar

Search

Crear

Ver planos

Espacios

Mi equipo de trabajo

Resumen

Acumulación de pendientes

Junta

Desarrollo

Cronología

Docs

Formas

Search ba...

BM

Filter

SCRUM-12

US08 - Registrar ventas de productos

Por hacer

8

SCRUM-27

US23 - Gestionar usuarios y permisos por rol

Por hacer

8

SCRUM-16

US12 - Registrador empleados

Por hacer

3

SCRUM-10

US06 - Generar alertas de stock mínimo

Por hacer

5

SCRUM-24

US20 - Generar reporte de inventario

Por hacer

5

SCRUM-25

US21 - Generar reporte de ventas y márgenes

Por hacer

5

SCRUM-14

US10 - Calcular margen de ganancia

Por hacer

8

SCRUM-15

US11 - Consultar historial de ventas

Por hacer

3

SCRUM-21

US17 - Clasificar gastos por categoría

Por hacer

2

SCRUM-22

US18 - Consultar gastos totales

Por hacer

3

SCRUM-27

US23 - Gestionar usuarios y permisos por rol

Por hacer

8

SCRUM-15

US11 - Consultar historial de ventas

Por hacer

3

SCRUM-21

US17 - Clasificar gastos por categoría

Por hacer

2

SCRUM-22

US18 - Consultar gastos totales

Por hacer

3

SCRUM-17

US13 - Definir turnos y horarios

Por hacer

5

SCRUM-18

US14 - Registrar y calcular nómina

Por hacer

8

SCRUM-19

US15 - Consultar costos de nómina

Por hacer

3

SCRUM-23

US19 - Generar reporte de gastos

Por hacer

5

SCRUM-26

US22 - Generar reporte de empleados y nómina

Por hacer

5

Crear

16°C

Prac. diseñado

6:35 p.m.

MATRIZ VALOR vs. ESFUERZO			
ESFUERZO →	BAJO	MEDIO	ALTO
VALOR ↓			
ALTO	● QUICK WINS (hacer primero) US01, US02, US05, US09, US16	● IMPORTANTES US03, US04, US06, US07, US08, US20, US21	● PROYECTOS GRANDES (planificar bien) US10, US23
MEDIO	● LLENAR huecos US11, US12, US15, US17, US18	● EVALUAR US13, US19	● CUIDADO (¿vale la pena ahora?) US14
BAJO	-	● BAJA PRIORIDAD US22	-

● QUICK WINS: dan mucho valor con poco esfuerzo → van primero en el backlog y son ideales para el Sprint 1.
 ● IMPORTANTES: valen la pena pero toman más trabajo → siguen en la fila.
 ● PROYECTOS GRANDES: alto valor pero alto esfuerzo → planificarlos con tiempo, no necesariamente en el Sprint 1.
 ● CUIDADO / BAJA PRIORIDAD: bajo retorno para el esfuerzo → quedan al final o incluso podrían no entrar.

Nota: Ejemplo de una historia de usuario con la descripción, los criterios de aceptación, la fecha (2 semanas +) del 23 de agosto al 6 de septiembre del 2026, las subtarear y el nivel de prioridad.

For you

Recent

Starred

Apps

Plans

Spaces

Recent

Mi equipo de trabajo

More spaces

Recommended

Create a roadmap

Filters

Dashboards

Loom

Teams

Goals

Projects

Customize sidebar

Search

+ Crear

Ver planos

BM

Espacios

Mi equipo de trabajo

Resumen

Acumulación de pendientes

Más 5

+

Q Lista de b...

BM

Filtro

...

SCRUM Sprint 1

23 ago – 6 sep (8 trabajos)

29 0 0

5

SCRUM-6 US02 ...

Por hacer

SCRUM-5 US01 ...

Por hacer

SCRUM-13 US0...

Por hacer

SCRUM-7 US03 ...

Por hacer

SCRUM-8 US04 ...

Por hacer

SCRUM-9 US05 ...

Por hacer

SCRUM-11 US0...

Por hacer

SCRUM-20 US16...

Por hacer

+ Crear

Objeto de trabajo de Jira

Detalles

Padre

Ninguno

Sprint

SCRUM Sprint 1

Prioridad

Más alto

Sellos

Debe

Fecha prevista de parto

Ninguno

Equipo

Ninguno

Fecha de inicio

Ninguno

Estimación de puntos de historia

2

Reportero

BM Brayhan Alexander Suárez ...

For you

Recent

Starred

Apps

Plans

Spaces

Recent

Mi equipo de trabajo

More spaces

Recommended

Create a roadmap

Filters

Dashboards

Loom

Teams

Goals

Projects

Customize sidebar

Search

+ Crear

Ver planos

BM

Espacios

Mi equipo de trabajo

Resumen

Acumulación de pendientes

Más 5

+

Q Lista de b...

BM

Filtro

...

SCRUM Sprint 1

23 ago – 6 sep (8 trabajos)

29 0 0

5

SCRUM-6 US02 ...

Por hacer

SCRUM-5 US01 ...

Por hacer

SCRUM-13 US0...

Por hacer

SCRUM-7 US03 ...

Por hacer

SCRUM-8 US04 ...

Por hacer

SCRUM-9 US05 ...

Por hacer

SCRUM-11 US0...

Por hacer

SCRUM-20 US16...

Por hacer

+ Crear

Objeto de trabajo de Jira

SCRUM-6 / SCRUM-28

US02 – Categorías del registro

Por hacer

Mejorar la subtask

Descripción

Diseñar formulario de categorías.

Crear entidad/modelo Categoría en Laravel.

Crear migración y tabla en MySQL.

Crear controlador y rutas (CRUD básico).

Crear validaciones de nombre único y obligatorio.

Realizar pruebas.

Integrar cambios a la rama principal.

Elementos de trabajo vinculados

Añadir elemento de trabajo vinculado

For you

Recent

Starred

Apps

Plans

Spaces

Recent

Mi equipo de trabajo

More spaces

Recommended

Create a roadmap

Filters

Dashboards

Loom

Teams

Goals

Projects

Customize sidebar

Search

+ Crear

Ver planos

BM

Espacios

Mi equipo de trabajo

Resumen

Acumulación de pendientes

Más 5

+

Q Search ba...

BM

Filter

...

SCRUM Sprint 1

Añadir fechas (8 elementos de trabajo)

29 0 0

5

SCRUM-6 US02 ...

Por hacer

SCRUM-5 US01 ...

Por hacer

SCRUM-13 US0...

Por hacer

SCRUM-7 US03 ...

Por hacer

SCRUM-8 US04 ...

Por hacer

SCRUM-9 US05 ...

Por hacer

SCRUM-11 US0...

Por hacer

SCRUM-20 US16...

Por hacer

+ Crear

Objeto de trabajo de Jira

Descripción

Como encargado de bodega, quiero crear y organizar categorías de productos (viveres, licores, cigarrillos, gaseosas/jugos), para que el inventario esté clasificado y sea más fácil de consultar.

Criterios de Aceptación

El sistema debe permitir crear una categoría con nombre único.

No debe permitir categorías con nombre repetido.

El campo nombre es obligatorio.

Debe mostrar confirmación cuando la categoría se registre correctamente.

Debe permitir listar todas las categorías existentes.

Nota: Las demas historias de usuario se veran reflejadas en el compartimiento de jira, adjunto link. <https://erp-software-architecture-viveres-ylicores.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog?atlOrigin=eyJpIjo7ZW1mZiMzU5NGFhYzYzM2NDNmNWE0NTQzNGExODQ3YjA2MjUiLCJwIjoiaWoiJ9>

2. Definition Of Done

Como equipo, se estableció la siguiente Definición de Terminado (Definition of Done), la cual aplica a toda historia de usuario del proyecto para considerarse formalmente finalizada:

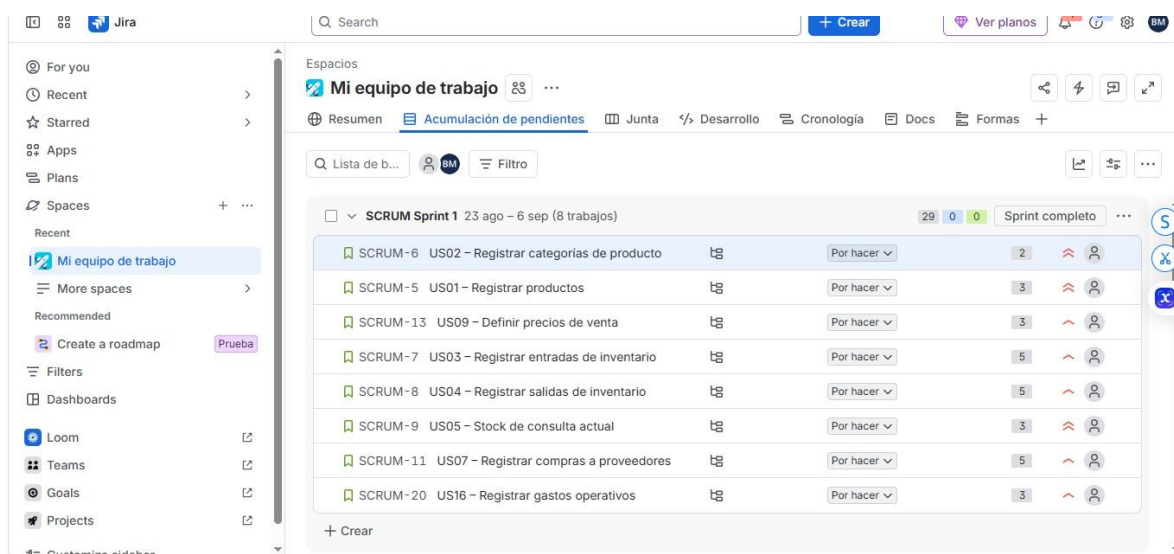
Una historia de usuario se considera **Done** cuando: (1) el código ha sido desarrollado siguiendo el patrón MVC de Laravel y los estándares de codificación acordados por el equipo; (2) se cumplen la totalidad de los criterios de aceptación definidos (mínimo 5 por historia); (3) se han ejecutado y aprobado los casos de prueba correspondientes; (4) no existen errores críticos ni bloqueantes conocidos; (5) el código ha sido integrado a la rama principal del repositorio en GitHub sin conflictos; (6) ha sido revisado por al menos un integrante distinto al autor (pull request aprobado); (7) la documentación técnica (Jira, README, comentarios en código) ha sido actualizada; y (8) ha sido validada en un ambiente de pruebas, no solo de forma local.

Esta definición fue acordada por consenso del equipo en la sesión de planificación del Sprint 0 y se aplicará de manera uniforme a lo largo de todos los sprints del proyecto.

3. Sprint Backlog del primer Sprint

Para el primer Sprint, con una duración de dos semanas, el equipo seleccionó del Product Backlog ocho historias de usuario correspondientes a la base funcional del sistema (registro de categorías, productos, precios, movimientos de inventario, compras a proveedores y gastos operativos), sumando un total de 29 Story Points, alcance considerado adecuado para la capacidad del equipo en el tiempo disponible.

Cada historia seleccionada fue especificada siguiendo la estructura "*Como <tipo de usuario>, quiero <objetivo>, para que <beneficio>*", se le asignaron mínimo 5 criterios de aceptación medibles y verificables, y se descompuso en tareas técnicas concretas (diseño, modelado, migración de base de datos, lógica de negocio, validaciones, pruebas e integración), las cuales fueron registradas como subtareas en Jira dentro de cada historia correspondiente. El detalle completo de las historias, sus criterios de aceptación y sus tareas se encuentra documentado en Jira y se adjunta como evidencia en el Anexo

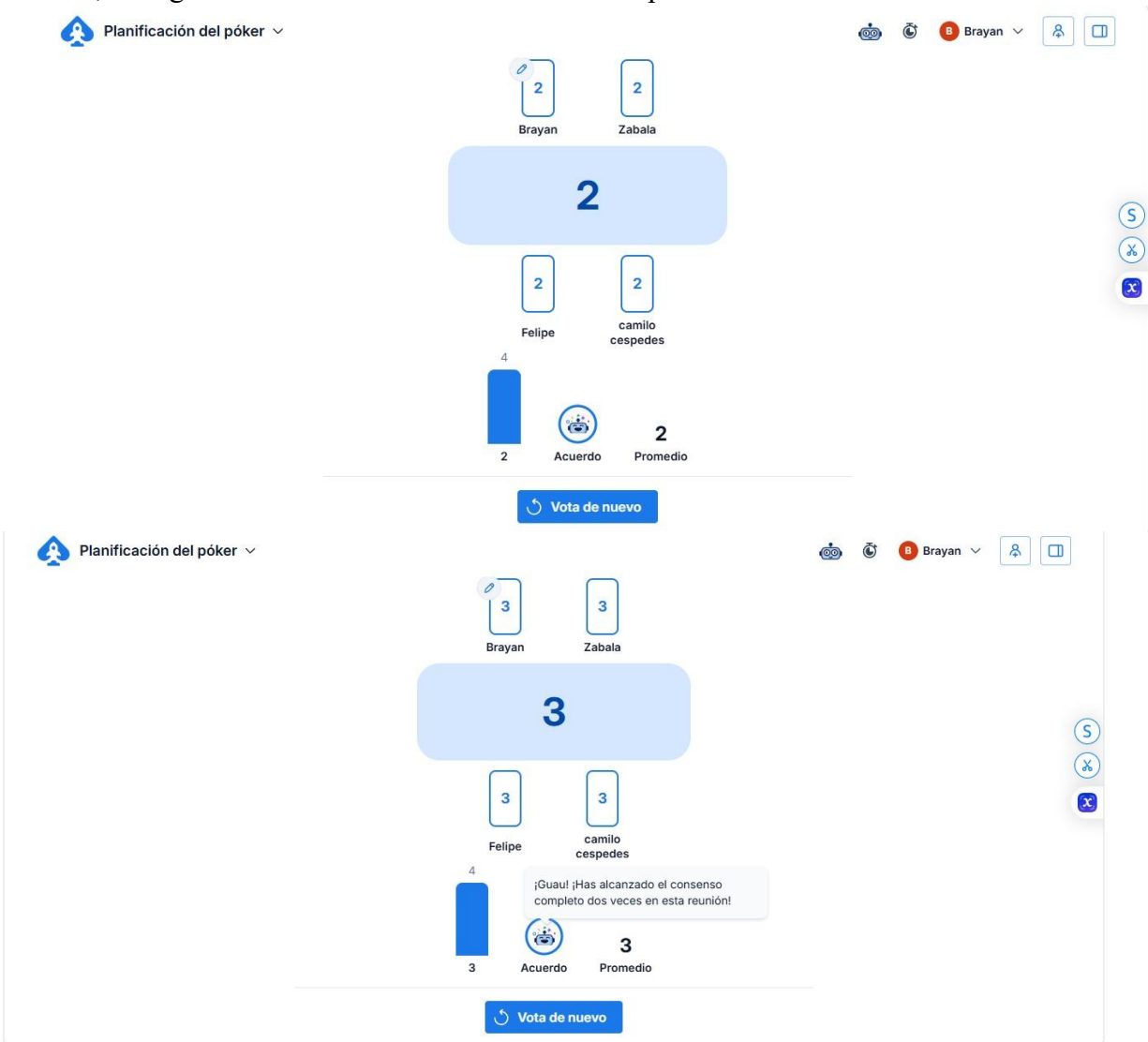


4. Estimar las User Stories (Planning Poker + Fibonacci)

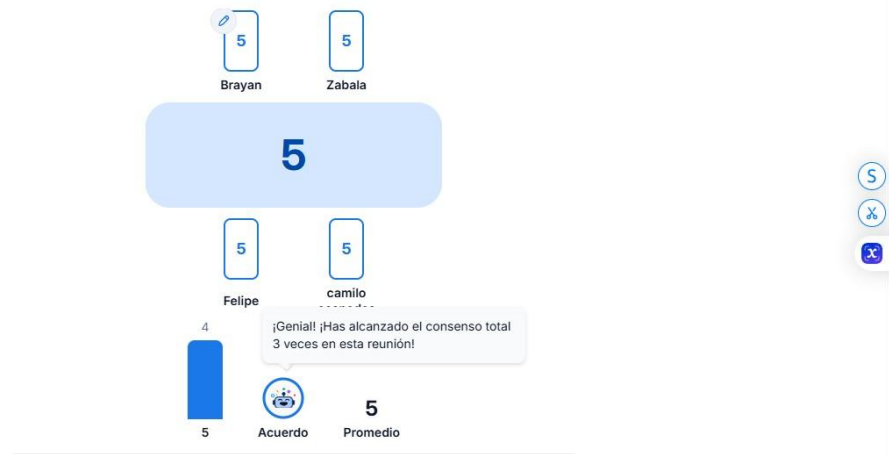
Para estimar la complejidad de las historias de usuario seleccionadas en el Sprint 1, el equipo realizó una sesión de **Planning Poker** utilizando la herramienta en línea *Planning Poker Online*, la cual permitió a cada integrante votar de manera independiente y simultánea utilizando la secuencia de Fibonacci (1, 2, 3, 5, 8, 13, 21).

Durante la sesión, se presentó cada historia de usuario del Sprint 1 al equipo y cada integrante emitió su voto sin conocer las estimaciones de los demás. Una vez revelados los votos, en los casos donde hubo diferencias significativas entre las estimaciones, los

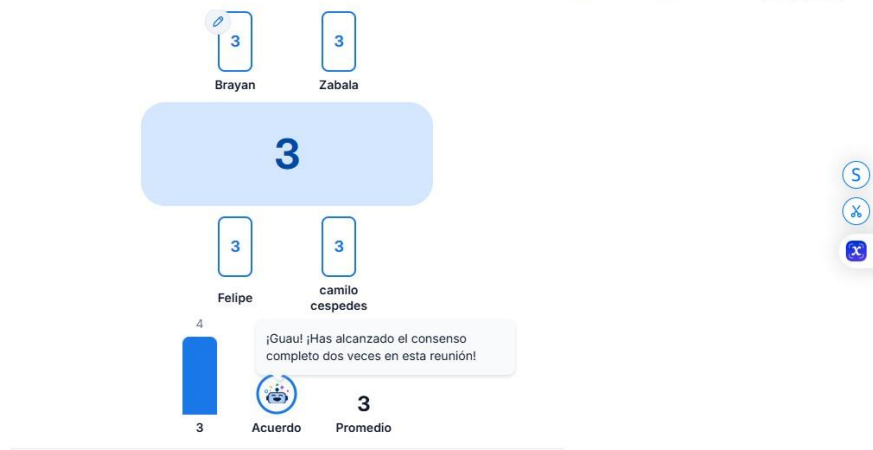
integrantes con los valores más extremos justificaron su punto de vista y se realizó una segunda ronda de votación hasta llegar a un consenso. Este proceso permitió que la estimación reflejara la percepción colectiva del equipo sobre la complejidad de cada historia, en lugar de basarse en el criterio de una sola persona.



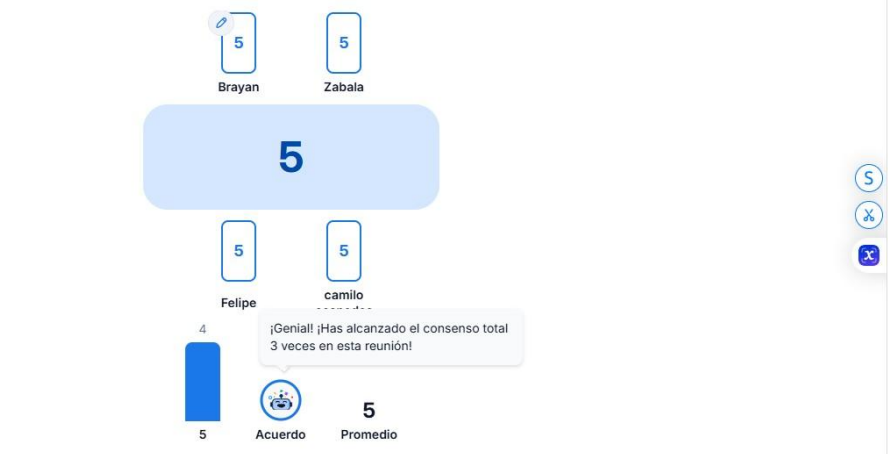




Vota de nuevo



Vota de nuevo



Vota de nuevo

El total estimado para el Sprint 1 fue de **29 Story Points**, cifra que fue considerada adecuada por el equipo dada su capacidad de trabajo durante las dos semanas de duración del sprint.

5. Plan de Pruebas

US02 – Registrar categorías de producto

ID	Caso	Resultado esperado
CP01	Registrar categoría con nombre válido	Se registra correctamente
CP02	Registrar categoría sin nombre	El sistema muestra error de campo obligatorio
CP03	Registrar categoría con nombre repetido	El sistema rechaza el registro
CP04	Registrar categoría con nombre muy largo (ej. 300 caracteres)	El sistema muestra error de validación
CP05	Consultar listado de categorías registradas	Muestra todas las categorías existentes

US01 – Registrar productos

ID	Caso	Resultado esperado
CP01	Registrar producto con todos los datos válidos	Se registra correctamente
CP02	Registrar producto sin nombre	El sistema muestra error
CP03	Registrar producto sin categoría asociada	El sistema rechaza el registro
CP04	Registrar producto con precio de compra negativo	El sistema muestra error de validación
CP05	Registrar producto con categoría inexistente	El sistema rechaza el registro
CP06	Registrar producto con datos válidos completos	Muestra mensaje de confirmación

US09 – Definir precios de venta

ID	Caso	Resultado esperado
CP01	Asignar precio de venta válido a un producto existente	Se guarda correctamente
CP02	Asignar precio de venta igual a cero	El sistema muestra error
CP03	Asignar precio de venta negativo	El sistema rechaza el valor
CP04	Asignar precio a un producto inexistente	El sistema muestra error
CP05	Actualizar el precio de venta de un producto ya definido	El precio se actualiza correctamente

US03 – Registrar entradas de inventario

ID	Caso	Resultado esperado
CP01	Registrar entrada con producto y cantidad válidos	Se registra correctamente y actualiza el stock
CP02	Registrar entrada de un producto inexistente	El sistema muestra error
CP03	Registrar entrada con cantidad igual a cero	El sistema rechaza el registro
CP04	Registrar entrada con cantidad negativa	El sistema muestra error de validación
CP05	Verificar que el stock aumente correctamente tras la entrada	El stock refleja la cantidad ingresada

US04 – Registrar salidas de inventario

ID	Caso	Resultado esperado
CP01	Registrar salida con stock suficiente	Se registra correctamente y descuenta el stock
CP02	Registrar salida con cantidad mayor al stock disponible	El sistema muestra error y rechaza el registro
CP03	Registrar salida con cantidad igual a cero	El sistema rechaza el registro
CP04	Registrar salida de un producto inexistente	El sistema muestra error
CP05	Verificar que el stock disminuya correctamente tras la salida	El stock refleja la cantidad descontada

US05 – Consultar stock actual

ID	Caso	Resultado esperado
CP01	Consultar stock de un producto existente correctamente	Muestra la cantidad actual
CP02	Consultar stock filtrando por categoría	Muestra solo los productos de esa categoría
CP03	Buscar un producto por nombre	Muestra el producto correspondiente
CP04	Buscar un producto que no existe	El sistema muestra "sin resultados"
CP05	Consultar producto con stock por debajo del mínimo	El sistema muestra indicador visual de alerta

US07 – Registrar compras a proveedores

ID	Caso	Resultado esperado
CP01	Registrar compra con proveedor, producto, cantidad y costo válidos	Se registra correctamente
CP02	Registrar compra sin proveedor	El sistema muestra error
CP03	Registrar compra con cantidad igual a cero	El sistema rechaza el registro
ID	Caso	Resultado esperado
CP04	Registrar compra con costo negativo	El sistema muestra error de validación
CP05	Verificar que se genere la entrada de inventario automáticamente	El stock del producto aumenta según la compra

US16 – Registrar gastos operativos

ID	Caso	Resultado esperado
CP01	Registrar gasto con concepto, monto y fecha válidos	Se registra correctamente
CP02	Registrar gasto sin concepto	El sistema muestra error
CP03	Registrar gasto con monto negativo	El sistema rechaza el registro
CP04	Registrar gasto con fecha futura	El sistema muestra error de validación
CP05	Registrar gasto con datos válidos completos	Muestra mensaje de confirmación

6. Definir responsabilidades del equipo

Aspecto	Encargado		
Técnico	Cespedes Correal	Merchan	Estrategia
Comunicación	Juan Zabala Ceron	Camilo	Diseño de la arquitectura del sistema, desarrollo del backend en Laravel y modelado de la base de datos en MySQL
Aseguramiento de calidad	David Moreno García	Felipe	Coordinación de reuniones del equipo, documentación en Jira/Confluence y seguimiento del avance de las historias
Apoyo técnico	Brayhan Alexander Suárez	Juan Camilo	Diseño y ejecución de casos de prueba, revisión de código mediante pull requests antes de integrar cambios
	Camilo		Desarrollo del frontend con Bootstrap, integración de módulos y despliegue del sistema

7. Cronograma del Sprint

Semana	Historia/Actividad	Responsable	Fecha estimada	Seguimiento
--------	--------------------	-------------	----------------	-------------

	US02 – Registrar categorías Cespedes Correal	24-25 ago	Daily / tablero 1 Jira
	US01 – Registrar productos Cespedes Correal	25-26 ago	Daily / tablero 1 Jira
1	US09 – Definir precios de venta Cespedes Correal	26 ago	Daily / tablero Jira
1	US03 – Registrar entradas de inventario Suárez Merchan	27-28 ago	Daily / tablero Jira
1	US04 – Registrar salidas de inventario Suárez Merchan	28 ago	Daily / tablero Jira
2	US05 – Consultar stock actual Cespedes Correal	31 ago	Daily / tablero Jira
2	US07 – Registrar compras a proveedores Suárez Merchan	31 ago - 01 sep	Daily / tablero Jira
2	US16 – Registrar gastos operativos Suárez Merchan	01 sep	Daily / tablero Jira
	Integración de módulos Suárez Merchan (apoyo técnico)	02 sep	Revisión en equipo 2
2	Ejecución de casos de prueba Moreno García	03 sep	Reporte de QA
2	Corrección de errores y cierre de sprint Todo el equipo	04 sep	Sprint Review + Retro

8. Documentación en Jira

https://erp-software-architecture-viveres-y-licores.atlassian.net/jira/software/projects/SCRUM/boards/1/timeline

Jira

Search

+ Create See plans Ask Roov

Spaces

Mi equipo de trabajo

Summary Backlog Board Development Timeline Docs Forms +

Search timeline Epic Label Status category

Work

August

Sprints

- SCRUM-36 Gestión de productos
- SCRUM-37 Gestión de categorías
- SCRUM-38 Gestión de movimientos de inventario
- SCRUM-39 Control y alertas de stock
- SCRUM-40 Gestión de compras
- SCRUM-41 Gestión de ventas
- SCRUM-42 Gestión de precios y márgenes
- SCRUM-43 Consulta de historial de ventas
- SCRUM-44 Gestión de empleados

Today Weeks Months Quarters

Spaces

Mi equipo de trabajo

Summary Backlog Board Development Timeline Docs Forms +

Search timeline Epic Label Status category

Work

August

Sprints

- SCRUM-45 Gestión de turnos y horarios
- SCRUM-46 Gestión de nómina
- SCRUM-47 Consulta de costos de nómina
- SCRUM-48 Registro de gastos
- SCRUM-49 Clasificación de gastos
- SCRUM-50 Consulta de gastos totales
- SCRUM-51 Edición de gastos
- SCRUM-52 Reporte de inventario
- SCRUM-53 Reporte de ventas y márgenes

Today Weeks Months Quarters

Search timeline Epic Label Status category

Work

August

Sprints

- SCRUM-48 Registro de gastos
- SCRUM-49 Clasificación de gastos
- SCRUM-50 Consulta de gastos totales
- SCRUM-51 Edición de gastos
- SCRUM-52 Reporte de inventario
- SCRUM-53 Reporte de ventas y márgenes
- SCRUM-54 Reporte de gastos
- SCRUM-55 Reporte de empleados y nómina

+ Create Epic

Today Weeks Months Quarters

Con el fin de cumplir con el requisito de contar con al menos una Feature y una User Story por integrante en cada módulo del sistema, el equipo organizó la documentación del proyecto en Jira mediante una estructura jerárquica de **Epics** (Features) y **Stories** (User Stories) vinculadas a estos.

Para cada uno de los cinco módulos del sistema (Inventario, Costos y Ventas, Empleados, Gastos, y Reportes y Análisis) se definieron cuatro Features, cada una asociada a una User Story específica y asignada a un integrante distinto del equipo, garantizando así una distribución equitativa del trabajo de documentación y desarrollo. Adicionalmente, se incorporó la historia US24 (edición y eliminación de gastos) para completar el cupo de historias del módulo de Gastos, y se reubicó la historia de reporte de gastos (US19) al módulo de Reportes y Análisis por coherencia funcional.

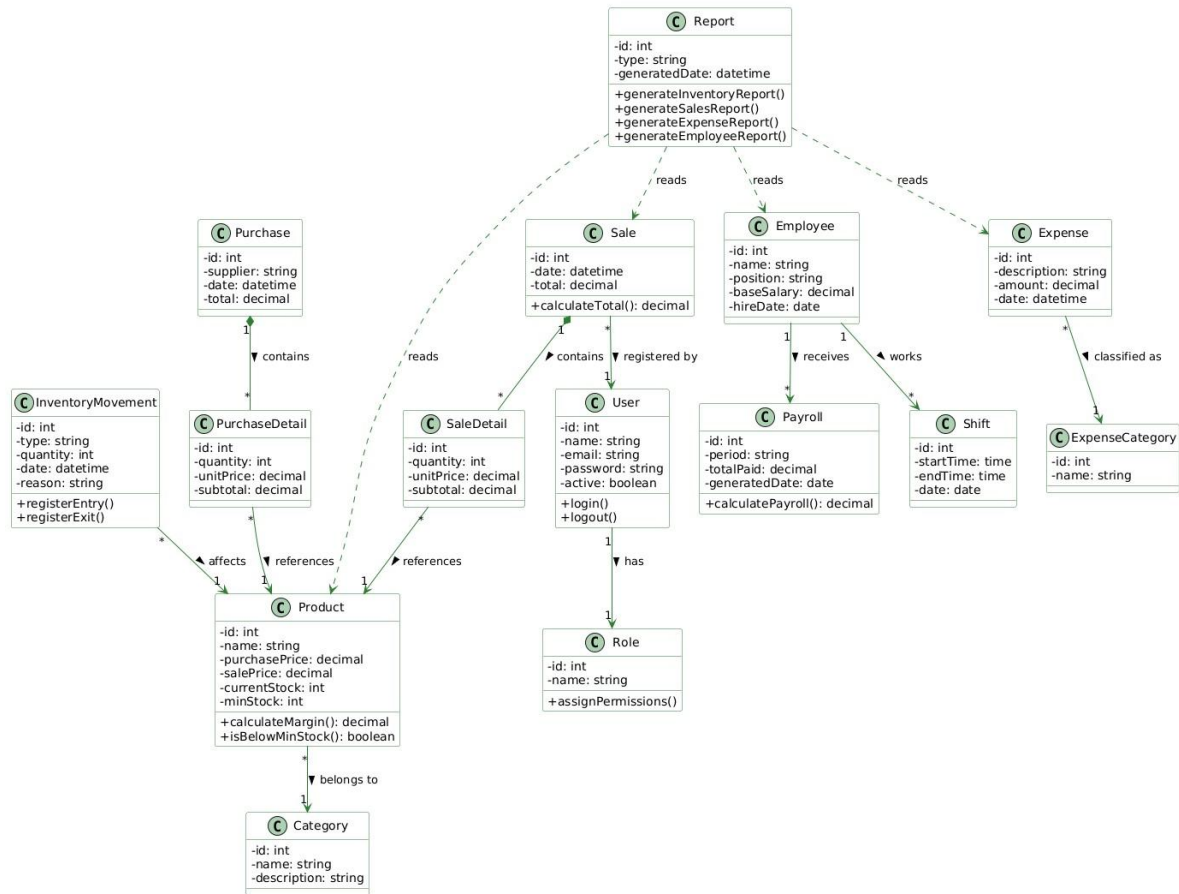
Toda esta documentación —Features, User Stories, criterios de aceptación, diagramas UML y árboles de descomposición funcional— se encuentra disponible tanto en el proyecto de Jira del equipo como en la página del proyecto alojada en GitHub Pages, cumpliendo con el requisito de trazabilidad y acceso a la documentación exigido por la guía.

Link de jira: <https://erp-software-architecture-viveres-y-licores.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog?atlOrigin=eyJpIjoizGYxOTgyMWRhZjhhNGY4NjgzYzhiMzFkMTQ2MWM4NzYiLCJwIjoiaoiJ9>

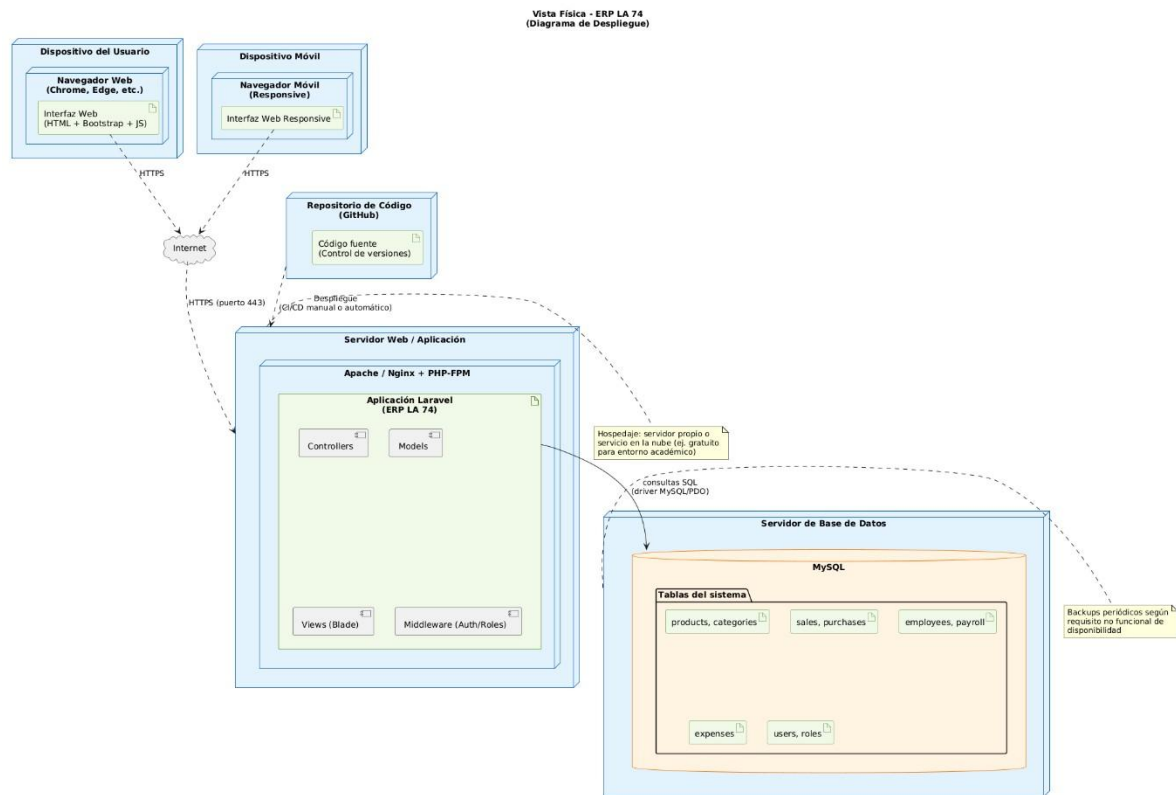
9. Diagramas UML

Vista Lógica

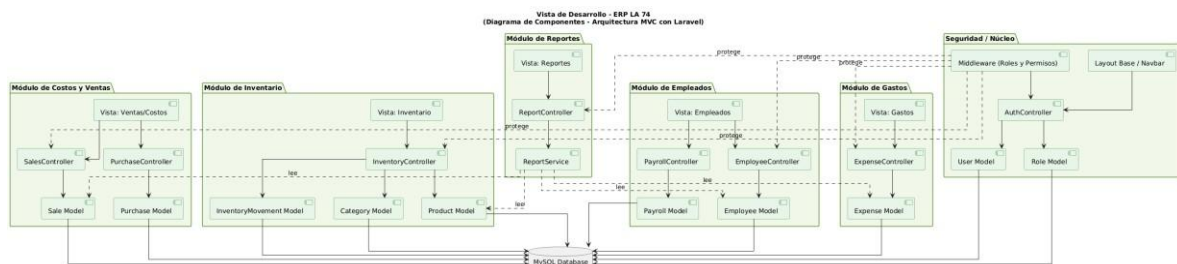
Vista Lógica - ERP LA 74
(Diagrama de Clases)



Vista Física



Vista de Desarrollo

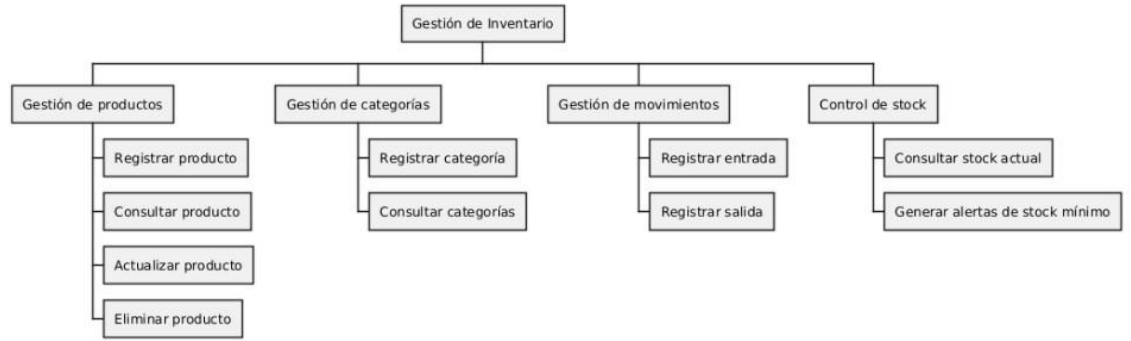


10. Arbol de descomposición funcional

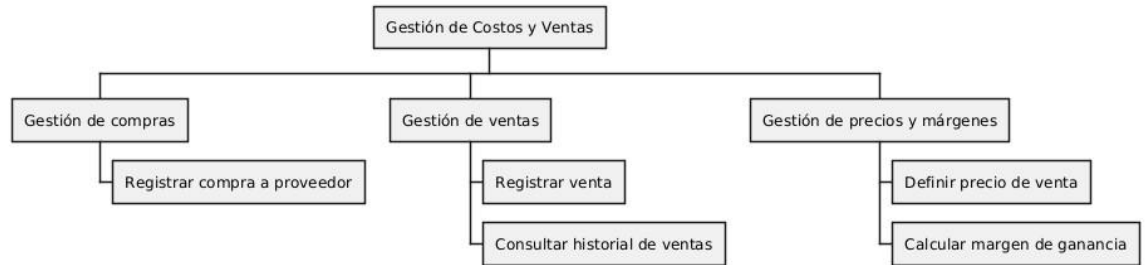
Arbol General



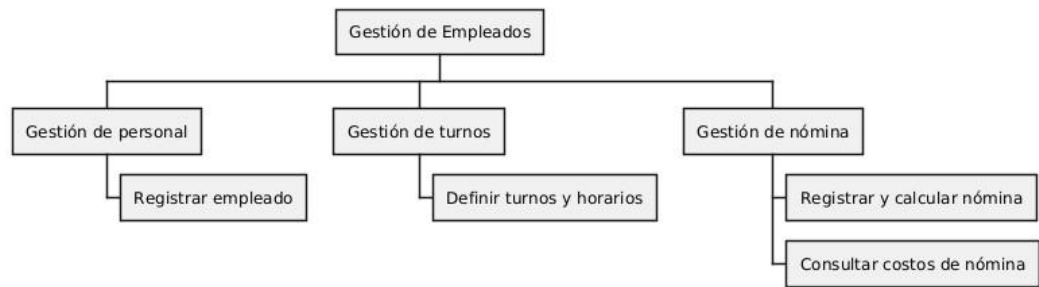
Modulo 1



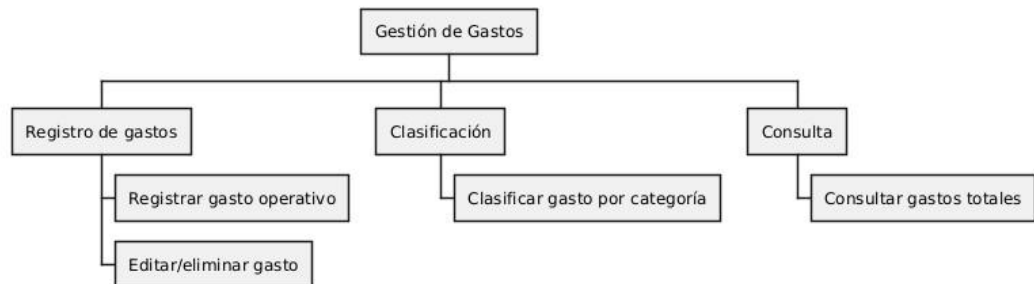
Modulo 2



Modulo 3



Modulo 4



Modulo 5



Link repositorio github: <https://github.com/jcamiloces/ERP-Software-Architecture-2026.git>